Dataset

```
1 !wget https://download.microsoft.com/download/3/E/1/3E1C3F21-ECDB-4869-8368
```

```
--2025-09-10 13:00:35--  https://download.microsoft.com/download/3/E/1/3E1C3F21-ECDB-4869-8368-6DEBA77
Resolving download.microsoft.com (download.microsoft.com)... 72.247.96.197, 2600:1406:5400:2a1::317f,
Connecting to download.microsoft.com (download.microsoft.com)|72.247.96.197|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 824887076 (787M) [application/octet-stream]
Saving to: ‘kagglecatsanddogs_5340.zip’

kagglecatsanddogs_5 100%[===================>] 786.67M  65.6MB/s    in 9.0s

2025-09-10 13:00:44 (87.4 MB/s) - ‘kagglecatsanddogs_5340.zip’ saved [824887076/824887076]
```

```
1 !unzip kagglecatsanddogs_5340.zip
```

```
inflating: PetImages/Dog/5856.jpg
inflating: PetImages/Dog/5857.jpg
inflating: PetImages/Dog/5858.jpg
inflating: PetImages/Dog/5859.jpg
inflating: PetImages/Dog/586.jpg
inflating: PetImages/Dog/5860.jpg
inflating: PetImages/Dog/5861.jpg
inflating: PetImages/Dog/5862.jpg
inflating: PetImages/Dog/5863.jpg
inflating: PetImages/Dog/5864.jpg
inflating: PetImages/Dog/5865.jpg
inflating: PetImages/Dog/5866.jpg
inflating: PetImages/Dog/5867.jpg
inflating: PetImages/Dog/5868.jpg
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import warnings
import os
import tqdm
import random
from keras.preprocessing.image import load_img
warnings.filterwarnings('ignore')
```

```
input_path = []
label = []

for class_name in os.listdir("PetImages"):
    for path in os.listdir("PetImages/"+class_name):
        if class_name == 'Cat':
            label.append(0)
        else:
            label.append(1)
        input_path.append(os.path.join("PetImages", class_name, path))
print(input_path[0], label[0])
```

```
PetImages/Cat/5180.jpg 0
```

```
df = pd.DataFrame()
df['images'] = input_path
df['label'] = label
df = df.sample(frac=1).reset_index(drop=True)
df.head()
```

	images	label
0	PetImages/Dog/12171.jpg	1
1	PetImages/Cat/8555.jpg	0
2	PetImages/Dog/7702.jpg	1
3	PetImages/Cat/6657.jpg	0
4	PetImages/Cat/1442.jpg	0

```
for i in df['images']:
    if '.jpg' not in i:
        print(i)
```

```
PetImages/Dog/Thumbs.db
PetImages/Cat/Thumbs.db
```

```
import PIL
l = []
for image in df['images']:
    try:
        img = PIL.Image.open(image)
    except:
        l.append(image)
l
```

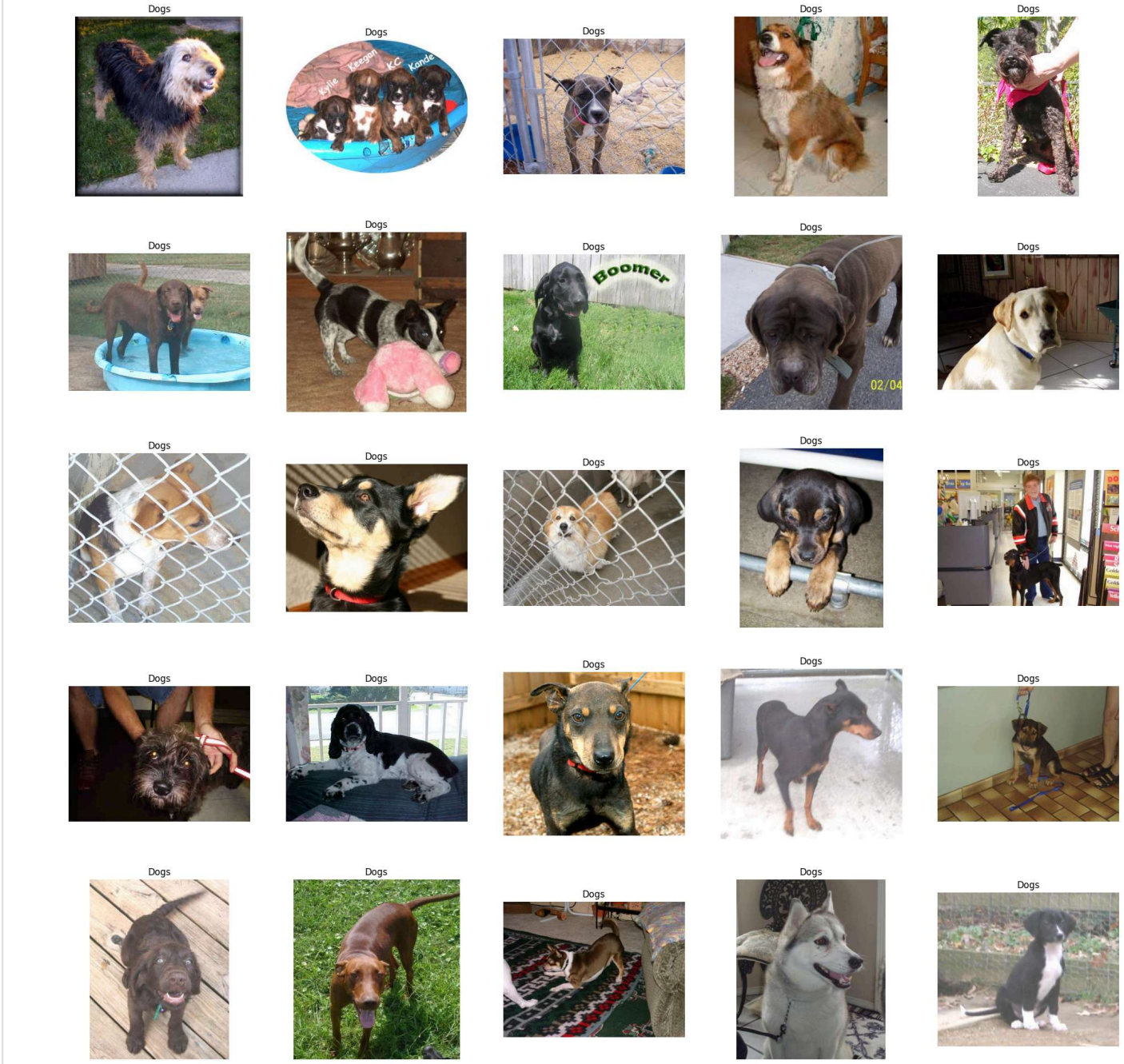
```
['PetImages/Dog/Thumbs.db',
 'PetImages/Dog/11702.jpg',
 'PetImages/Cat/Thumbs.db',
 'PetImages/Cat/666.jpg']
```

```
# delete db files
df = df[df['images']!='PetImages/Dog/Thumbs.db']
df = df[df['images']!='PetImages/Cat/Thumbs.db']
df = df[df['images']!='PetImages/Cat/666.jpg']
df = df[df['images']!='PetImages/Dog/11702.jpg']
len(df)
```

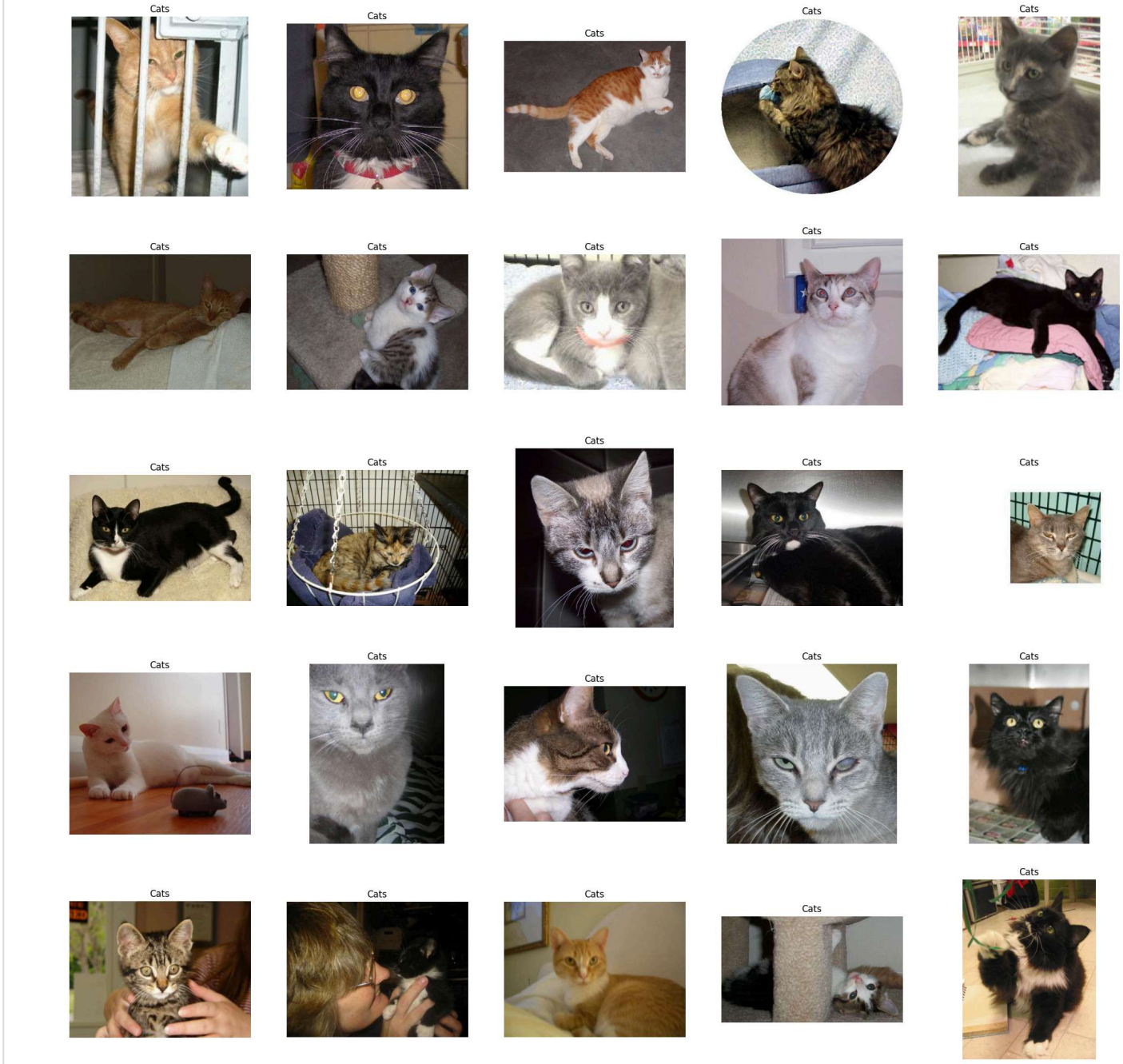
```
24998
```

Exploratory Data Analysis

```
# to display grid of images
plt.figure(figsize=(25,25))
temp = df[df['label']==1]['images']
start = random.randint(0, len(temp))
files = temp[start:start+25]

for index, file in enumerate(files):
    plt.subplot(5,5, index+1)
    img = load_img(file)
    img = np.array(img)
    plt.imshow(img)
    plt.title('Dogs')
    plt.axis('off')
```

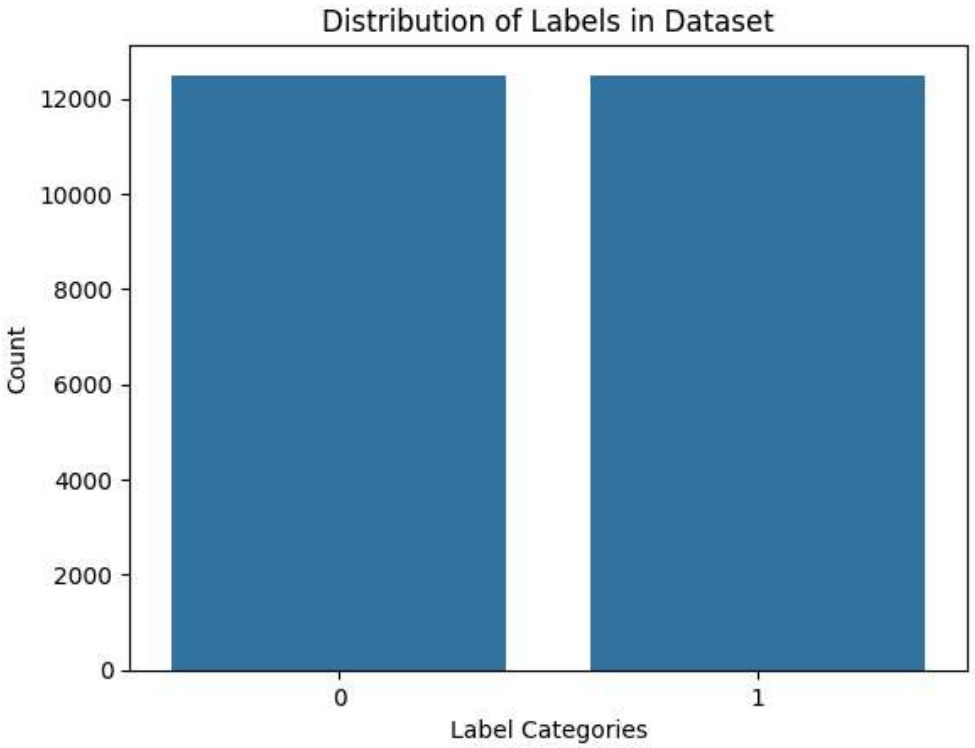



```
# to display grid of images
plt.figure(figsize=(25,25))
temp = df[df['label']==0]['images']
start = random.randint(0, len(temp))
files = temp[start:start+25]

for index, file in enumerate(files):
    plt.subplot(5,5, index+1)
    img = load_img(file)
    img = np.array(img)
    plt.imshow(img)
    plt.title('Cats')
    plt.axis('off')
```




```
 1 import seaborn as sns
 2 import matplotlib.pyplot as plt
 3
 4 # Create a count plot for the 'label' column
 5 sns.countplot(x=df['label'])
 6
 7 # Add labels and title
 8 plt.xlabel("Label Categories")
 9 plt.ylabel("Count")
10 plt.title("Distribution of Labels in Dataset")
11
12 # Show the plot
```

```
plt.show()

```



Create DataGenerator for the Images

```
df['label'] = df['label'].astype('str')
```

```
df.head()
```

	images	label
0	PetImages/Dog/12171.jpg	1
1	PetImages/Cat/8555.jpg	0
2	PetImages/Dog/7702.jpg	1
3	PetImages/Cat/6657.jpg	0
4	PetImages/Cat/1442.jpg	0

```
# input split
from sklearn.model_selection import train_test_split
train, test = train_test_split(df, test_size=0.2, random_state=42)
```

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator

train_generator = ImageDataGenerator(
    rescale = 1./255,  # normalization of images
    rotation_range = 40, # augmention of images to avoid overfitting
    shear_range = 0.2,
```

```
    zoom_range = 0.2,
    horizontal_flip = True,
    fill_mode = 'nearest'
)

val_generator = ImageDataGenerator(rescale = 1./255)

train_iterator = train_generator.flow_from_dataframe(
    train,
    x_col='images',
    y_col='label',
    target_size=(128,128),
    batch_size=512,
    class_mode='binary'
)

val_iterator = val_generator.flow_from_dataframe(
    test,
    x_col='images',
    y_col='label',
    target_size=(128,128),
    batch_size=512,
    class_mode='binary'
)
```

```
Found 19998 validated image filenames belonging to 2 classes.
Found 5000 validated image filenames belonging to 2 classes.
```

model Creation

```
from keras import Sequential
from keras.layers import Conv2D, MaxPool2D, Flatten, Dense

model = Sequential([
                    Conv2D(16, (3,3), activation='relu', input_shape=(128,1
                    MaxPool2D((2,2)),
                    Conv2D(32, (3,3), activation='relu'),
                    MaxPool2D((2,2)),
                    Conv2D(64, (3,3), activation='relu'),
                    MaxPool2D((2,2)),
                    Flatten(),
                    Dense(512, activation='relu'),
                    Dense(1, activation='sigmoid')
])
```

```
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accur
model.summary()
```

```
Model: "sequential_2"
```

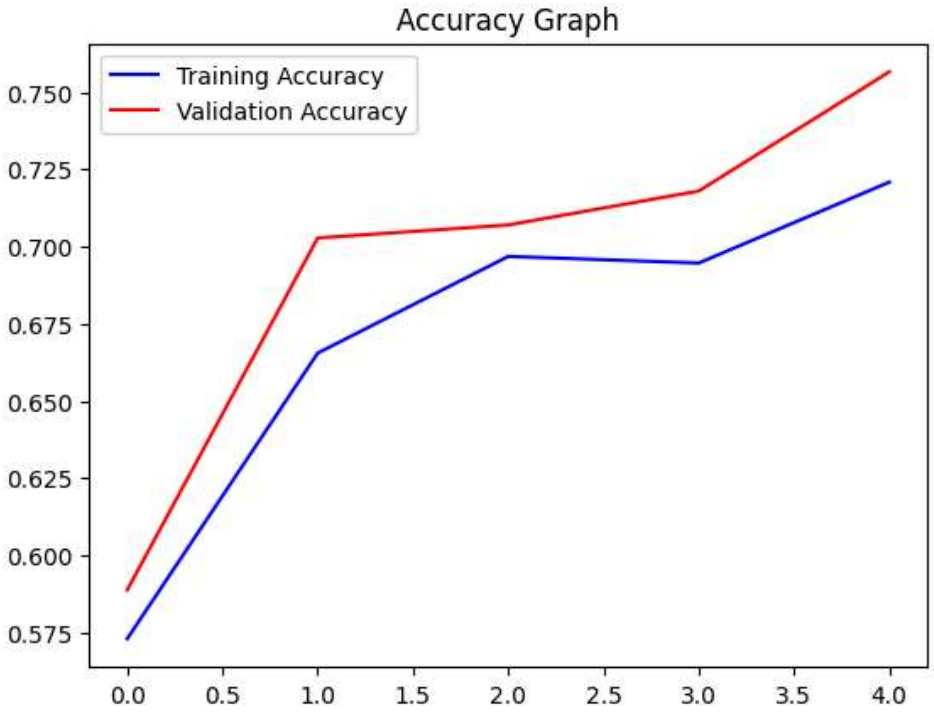
Layer (type)	Output Shape	Param #
conv2d_6 (Conv2D)	(None, 126, 126, 16)	448
max_pooling2d_6 (MaxPooling2D)	(None, 63, 63, 16)	0
conv2d_7 (Conv2D)	(None, 61, 61, 32)	4,640
max_pooling2d_7 (MaxPooling2D)	(None, 30, 30, 32)	0
conv2d_8 (Conv2D)	(None, 28, 28, 64)	18,496
max_pooling2d_8 (MaxPooling2D)	(None, 14, 14, 64)	0
flatten_2 (Flatten)	(None, 12544)	0
dense_4 (Dense)	(None, 512)	6,423,040
dense_5 (Dense)	(None, 1)	513

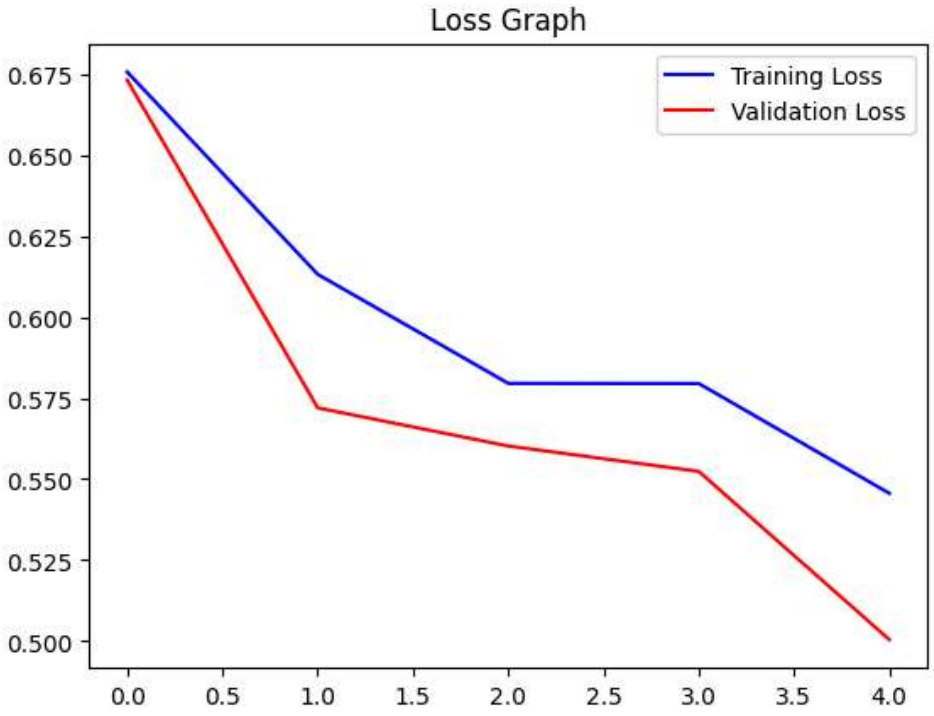
```
Total params: 6,447,137 (24.59 MB)
Trainable params: 6,447,137 (24.59 MB)
Non-trainable params: 0 (0.00 B)
```

```
history = model.fit(train_iterator, epochs=5, validation_data=val_iterator)
```

```
Epoch 1/5
40/40 ━━━━━━━━━━━━━━━━━━━━ 116s 3s/step - accuracy: 0.5377 - loss: 0.6877 - val_accuracy: 0.5888 - val
Epoch 2/5
40/40 ━━━━━━━━━━━━━━━━━━━━ 109s 3s/step - accuracy: 0.6498 - loss: 0.6265 - val_accuracy: 0.7028 - val
Epoch 3/5
40/40 ━━━━━━━━━━━━━━━━━━━━ 107s 3s/step - accuracy: 0.6865 - loss: 0.5897 - val_accuracy: 0.7070 - val
Epoch 4/5
40/40 ━━━━━━━━━━━━━━━━━━━━ 108s 3s/step - accuracy: 0.7044 - loss: 0.5691 - val_accuracy: 0.7180 - val
Epoch 5/5
40/40 ━━━━━━━━━━━━━━━━━━━━ 108s 3s/step - accuracy: 0.7133 - loss: 0.5556 - val_accuracy: 0.7566 - val
```

```
acc = history.history['accuracy']
val_acc = history.history['val_accuracy']
epochs = range(len(acc))

plt.plot(epochs, acc, 'b', label='Training Accuracy')
plt.plot(epochs, val_acc, 'r', label='Validation Accuracy')
plt.title('Accuracy Graph')
plt.legend()
plt.figure()

loss = history.history['loss']
val_loss = history.history['val_loss']
plt.plot(epochs, loss, 'b', label='Training Loss')
plt.plot(epochs, val_loss, 'r', label='Validation Loss')
plt.title('Loss Graph')
plt.legend()
plt.show()
```





Test with Real Image

```
image_path = "/content/Dog_Breeds.jpg" # path of the image
img = load_img(image_path, target_size=(128, 128))
img = np.array(img)
img = img / 255.0 # normalize the image
img = img.reshape(1, 128, 128, 3) # reshape for prediction
pred = model.predict(img)
if pred[0] > 0.5:
    label = 'Dog'
```

```
else:
    label = 'Cat'
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 623ms/step
Dog
```

```
image_path = "/content/images__C.jpg" # path of the image
img = load_img(image_path, target_size=(128, 128))
img = np.array(img)
img = img / 255.0 # normalize the image
img = img.reshape(1, 128, 128, 3) # reshape for prediction
pred = model.predict(img)
if pred[0] > 0.5:
    label = 'Dog'
else:
    label = 'Cat'
print(label)
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step
Cat
```

```
import pickle

# Assume your model is in a variable called 'model'
with open("model.pkl", "wb") as f:
    pickle.dump(model, f)

```

```
with open("model.pkl", "rb") as f:
    model = pickle.load(f)

```

```
# Save model
model.save("cat_dog_model.h5")

# Load model
from tensorflow.keras.models import load_model
model = load_model("cat_dog_model.h5")

```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(m
WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. `model.compile_m
```